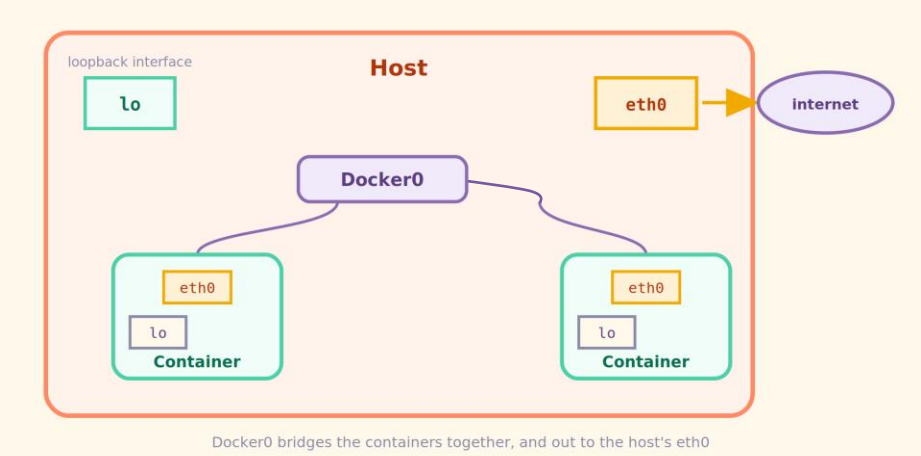
Docker Networking

1. How does a Container actually talk?

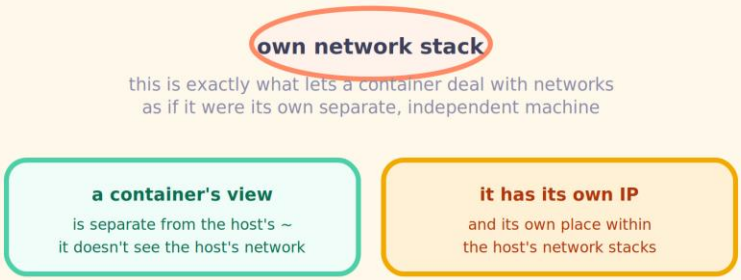
How exactly does a Container communicate with another Container, or reach the internet?



Docker0 bridges the containers together, and out to the host's eth0

On the Host itself, there's a loopback interface (lo) and an eth0 that connects out to the internet. Docker0 sits in between and connects to each Container, and every Container has its own eth0 and its own lo inside it.

Each container has its own network stack



A container's view is completely disconnected from the host's own network — it can't see what the host sees. It has its own IP, and its own distinct place within the host's network stacks.

How Docker connects everything by default

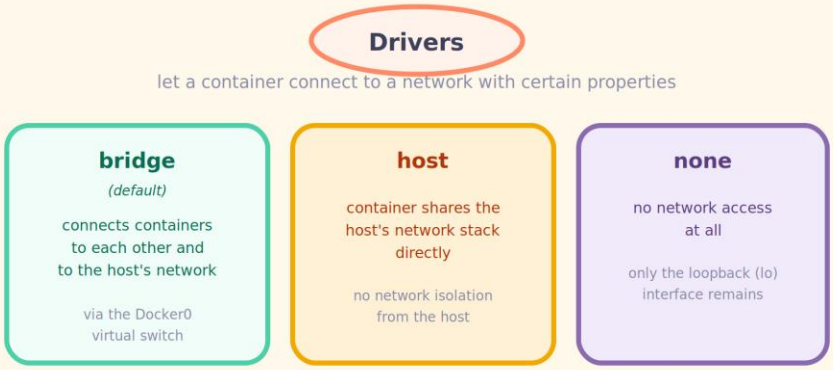
By default, Docker makes a container able to connect to others by creating a Docker bridge. This bridge connects all containers to each other, and also allows them to reach the internet through the host — some also describe this as going through the host's network card.

This all happens via Docker0, which acts as a virtual switch.

---

2. Drivers: how does Docker manage networks?

Drivers are what let a container connect to a network with certain characteristics or properties. There are 3 main types:



- bridge (the default) — connects containers to each other and to the host's network, through the Docker0 virtual switch.
- host — the container shares the host's network stack directly, with no isolation between them.
- none — the container has no network connectivity at all; only its loopback (lo) interface remains.

~ if we don't specify a driver when creating a container, Docker uses bridge by default, since that's what Docker's own bridge network is built on.

---

3. Useful commands

A quick reference for inspecting and managing container networking:

Inspecting interfaces

```
ip link show ~ lists network interfaces
ip addr show ~ lists IP addresses
```

Running containers with networking options

```
docker container run --add-host name:IP image ~ adds a custom host entry
docker container run --network driver image ~ runs with a specific driver
```

Managing networks

```
docker network create --subnet address name ~ creates a network with a given subnet
docker network connect network-name container ~ attaches a running container
docker network disconnect network-name container ~ detaches a container
docker network create --internal network-name ~ creates a network with no outside access
```